PROVEIT / DIOPHANTINE CONSISTENCY

Additional Lean Proof Simplifications

Checked proposals beyond the incoming packets

Prepared for Vladimir Reshetnikov
26 October 2026

Main result. A proposed patch changes nine existing modules and adds one shared module. It saves **106 physical lines**, counting every added helper, wrapper, comment, and blank line. Its main mathematical improvements preserve signed rational roots during descent, reuse fundamental-unit classification for small Pell ordinates, and compose sign-ambiguous residues. Its representation improvements consume existing certificate equivalences without changing the represented data.

Novelty scope. The comparison covers the seven ZIP archives present in `docs/incoming`, plus the earlier packet referenced by that directory's README, at the exact repository snapshot below. This is a claim of additional repository proposals relative to that inspected material, not a claim of priority for the underlying mathematics or Lean techniques.

Reproducibility. The accompanying archive contains the LaTeX and PDF, the patch and complete proposed modules, original-versus-proposed metrics, nine complete-value equality checks for representation constructors, Lean validation logs, source and archive hashes, a detailed exclusion audit, and instructions for replay. An additional nine-line permutation proposal is kept separate because its full repository client was not compiled.

Repository commit:

`7fa06d530df54d4aeb2e18027409df082234c69e`

Lean: `4.32.0` mathlib: `81a5d257c8e410db227a6665ed08f64fea08e997`

Contents

1 Scope, evidence, and results 2

2 Preserve the exact rational root during integral descent 3

3 A uniform small-ordinate criterion for Pell minimality 5

4 Reuse the existing odd-index theorem 6

5 Compose congruence up to sign as a relation 7

6 Derive inverse pairing graphs by existential projection 8

7 Consume the Boolean certificate before constructing a conditional 9

8 Optional: recognize the physical permutation as a positive tail 10

9 Validation and its limits 11

10 Adoption order and package guide 12

11 Further research questions 13

A Novelty comparison at a glance 14

B Primary source anchors 15

1 Scope, evidence, and results

The request is to shorten and simplify existing proofs in `Computability/DiophantineConsistency`, while avoiding proposals already present in `docs/incoming`. All source locations and comparisons in this report refer to commit `7fa06d530df54d4aeb2e18027409df082234c69e`. The live branch can subsequently acquire further proposals.

The main library directory contains 391 Lean files and 71,601 physical lines in the inspected snapshot. Including its entry point and checking directory, the broader Lean directory contains 574 Lean files. These counts establish the scope of the source survey; they do not mean every theorem was individually reproved or every module rebuilt.

The incoming audit covers seven archives containing 1,990 members. It identifies 177 report/proposal artifacts and separately reviews the 60-file placed packet referenced by the incoming README. Bundled source snapshots were used to understand earlier recommendations, but were not mistaken for recommendations themselves. The resulting exclusion map has 57 families, with archive names, member paths, section locations, and prior claimed validation status. The complete map is supplied as `audit/novelty_audit.md` and JSON.

1.1 What was deliberately excluded

Earlier packets already propose core list-lookup reuse, recursor congruence, assignment transport and witness-block splitting, generic sum-of-squares lemmas, shared degree and cost proofs, scanner simulation, circuit power certificates, coefficient envelopes, partial-decoder injectivity, structured source facts, modular cast cleanup, and several finite-tuple interfaces. Those proposals are not presented again as contributions.

Some techniques below use general library facts that earlier reports also mention. The contribution must therefore be identified more precisely than using `simp` or factoring a helper. For example, the pairing proposal uses an inverse coding relation to remove particular existential coordinates. It does not repeat the earlier recommendation to split the witness block of `Rep.conjunction`. Likewise, the Boolean proposal consumes an existing quadratic zero-certificate equivalence; it does not redesign a scanner's numeric Boolean interface.

No matching proposal was found in the inspected reports, notes, candidate code, or placed packet for the seven directions in this report. This is a bounded audit result. It does not rule out a similar idea elsewhere in the repository's history, in material not present at this snapshot, or in the wider Lean community.

1.2 Measured source changes

The main patch keeps all pre-existing declaration signatures in the changed files. Its representation changes additionally have complete-value equality checks, discussed in Section 9.

Module or group	Before	After	Saved
NearSquareFinalizer	255	251	4
MarginFinalizer	177	135	42
PairingRepresentation	88	94	−6
QuadraticNatPairing	158	139	19
QuadraticTreeCodeGraphs	150	136	14
QuadraticBoolean	136	126	10
NativePellBootstrap	685	677	8
NativePellStepDown	248	196	52
SignedPellResidue	102	96	6
SignedModEq (new)	0	43	−43
Main patch	**1,999**	**1,893**	**106**
WangTiles942Permutation (optional)	264	255	9

These are physical source lines, including comments and blank lines, computed from the actual delivered files. Negative entries charge shared infrastructure to the change that introduces it. The 106-line result is a 5.3% reduction in the selected production source, not a 5.3% reduction in the entire library. The optional nine lines are not included in the main total.

Source length is one measure. The larger improvement is that several proofs now expose the mathematical reason for the conclusion: a denominator equal to one, growth of Pell powers, transitivity of a signed residue relation, or an already-proved certificate equivalence. No compilation-speed or proof-term-size improvement is claimed.

2 Preserve the exact rational root during integral descent

2.1 The avoidable loss of information

Both `NearSquareFinalizer.lean` and `MarginFinalizer.lean` reconstruct rational numbers r, s satisfying

$$r^2 = a, \qquad s^2 = b, \qquad \frac{\theta}{\omega} = r + ws.$$

The existing proofs then use `Rat.isSquare_intCast_iff` to obtain *some* integer roots r_0, s_0. They must recover the relationship between the roots in the linear equation and those new witnesses:

$$r = \pm r_0, \qquad s = \pm s_0.$$

This leads to four combinations, each carrying essentially the same final cast and multiplication argument. The original evidence was more informative than the intermediate `IsSquare` statement.

The replacement retains each rational witness. A rational number whose nonzero natural power is an integer already has denominator one. Its numerator is therefore the exact signed integer needed in the linear equation.

Lemma 1. *Let $q \in \mathbb{Q}$, $z \in \mathbb{Z}$, and $n \in \mathbb{N} \setminus \{0\}$. If $q^n = z$, then $q = \mathrm{num}(q)$, with the right side embedded in $\mathbb{Q}$.*

Proof. Write q in reduced form. The rational denominator API gives

$$\mathrm{den}(q^n) = \mathrm{den}(q)^n.$$

An embedded integer has denominator one. Hence $\mathrm{den}(q)^n = 1$, which forces $\mathrm{den}(q) = 1$ because $n \neq 0$. The numerator identity follows. This reasoning preserves the sign of q. □

The checked implementation is short:

```
theorem rat_intCast_num_of_pow {q : ℚ} {z : ℤ} {n : ℕ}
    (hn : n ≠ 0) (h : q ^ n = (z : ℚ)) :
    (q.num : ℚ) = q := by
  apply Rat.coe_int_num_of_den_eq_one
  simpa [hn] using congrArg Rat.den h
```

The proof uses the denominator identities for powers and embedded integers, the natural-power equation for one, and `Rat.coe_int_num_of_den_eq_one`; see mathlib source and the Lean rational-number definitions included in the pinned toolchain. The hypothesis $n \neq 0$ is essential: every rational number has zeroth power one.

2.2 The client proof keeps both signs automatically

After the existing rational reconstruction, the new finalizer proof uses:

```
have hrInt := rat_intCast_num_of_pow (by decide : 2 ≠ 0) hr
have hsInt := rat_intCast_num_of_pow (by decide : 2 ≠ 0) hs
rw [← hrInt] at hr hsum
rw [← hsInt] at hs hsum
refine ⟨r.num, s.num, ?_, ?_, ?_⟩
· exact_mod_cast hr
· exact_mod_cast hs
· exact_mod_cast
    ((div_eq_iff hoQ).mp hsum).trans (mul_comm _ _)
```

The integer witnesses are the numerators of the very roots that already satisfy the sum equation. There is no sign search. Negative roots, zero radicands where otherwise permitted, and either sign of the nonzero weight remain covered.

This is more specific than a general cast cleanup. It replaces an intermediate existence assertion about an integer square root by integrality of the particular rational root. The latter is exactly the information the subsequent linear relation needs.

2.3 Put the general separated theorem below both clients

The two files also contain near-duplicate root reconstruction proofs. Their actual common hypotheses are

$$\omega \neq 0, \qquad w \neq 0, \qquad a \neq w^2 b, \qquad \operatorname{norm}(\theta, \omega, a, w, b) = 0.$$

These suffice to reconstruct integer r, s with

$$r^2 = a, \qquad s^2 = b, \qquad \theta = \omega(r + ws).$$

The patch places this separated theorem in `NearSquareFinalizer`, where the norm and rational reconstruction already live. `MarginFinalizer.integer_roots_of_separated_norm_zero` keeps its original public name and signature as a direct wrapper.

The earlier bounded form in `NearSquareFinalizer` now obtains separation from its existing hypotheses $w > 0$, $a < w^2$, $b > 0$. Since b is an integer, $b \geq 1$, so $w^2 b \geq w^2 > a$. Its proof becomes an application of the general theorem plus this elementary inequality.

This relocation follows the actual dependency direction. It adds no import to either file and no cycle. It also preserves both original APIs. The norm and final-polynomial definitions are unchanged, and the complete revised files compile together. The net reduction is **46 lines**, including the power lemma, generalized theorem, specialization, and wrapper.

3 A uniform small-ordinate criterion for Pell minimality

3.1 A stronger lemma replaces a discriminant-specific argument

In `NativePellBootstrap.lean`, `firstUnit_fundamental` concerns

$$d = Q(Q+1), \qquad (x, y) = (2Q+1, 2), \qquad Q > 0.$$

The existing proof analyzes a competing solution, bounds its squared ordinate, splits its ordinate into $-1, 0, 1$, and excludes those possibilities using adjacent-square inequalities. That elementary proof is valid. The following general fact provides a shorter route with existing mathlib APIs.

Proposition 1. *For any integer d, let $a = (x, y)$ solve $x^2 - dy^2 = 1$. If $x > 1$ and $0 < y \leq 3$, then a is fundamental. The bound three cannot be increased to four.*

Proof. The existence of a solution with $x > 1$ implies that d is positive and nonsquare. Let $f = (u, v)$ be the fundamental positive solution. The classification theorem expresses $a = f^n$ for a natural n. Positive y excludes $n = 0$.

If $n \geq 2$, monotonicity of the ordinate along powers gives

$$a.y \geq (f^2).y = 2uv.$$

Fundamentality gives the integer bounds $u \geq 2$ and $v \geq 1$, hence $a.y \geq 4$, a contradiction. Thus $n = 1$.

For sharpness, $d = 3$ has the solution $(7, 4)$. It is not fundamental because $(2, 1)$ is a positive solution with a smaller first coordinate. □

The checked general theorem is:

```
theorem isFundamental_of_y_le_three
    {d : ℤ} {a : Pell.Solution₁ d}
    (hx : 1 < a.x) (hy : 0 < a.y) (hsmall : a.y ≤ 3) :
    Pell.IsFundamental a := by
  obtain ⟨f, hf⟩ := Pell.IsFundamental.exists_of_not_isSquare
    (Pell.Solution₁.d_pos_of_one_lt_x hx)
    (Pell.Solution₁.d_nonsquare_of_one_lt_x hx)
  obtain ⟨n, rfl⟩ := hf.eq_pow_of_nonneg (by omega) hy.le
  have hn : n < 2 := by
    by_contra hn
    have h2 := hf.y_strictMono.monotone
      (show (2 : ℤ) ≤ n by omega)
    simp only [zpow_natCast, zpow_ofNat, pow_two,
```

```
      Pell.Solution₁.y_mul] at h2
    have hfx := hf.1
    have hfy := hf.2.1
    nlinarith only [h2, hsmall, hfx, hfy,
      mul_nonneg (show (0 : ℤ) ≤ f.x - 2 by omega)
        (show (0 : ℤ) ≤ f.y - 1 by omega)]
  interval_cases n
  · simp at hy
  · simpa using hf
```

The exact source file formats two of these expressions on longer lines; the displayed version wraps them for readability. The source metrics count the delivered file, not this presentation.

3.2 Benefits and tradeoff

The original first-unit theorem becomes a two-line proof body invoking the criterion. Its nonblank proof body decreases from 30 lines to 2; charging the new helper's 14 nonblank proof-body lines leaves a reduction of 14 by that measure. Counting the complete production file, including documentation and all declarations, gives **685 to 677 lines**, a saving of eight.

The new theorem is reusable for any positive Pell solution with ordinate one, two, or three. Its sharpness witness is also checked in the standalone artifact. This is a mathematical strengthening of the reusable interface, rather than moving the old case analysis into a differently named lemma.

The tradeoff is explicit: the new proof uses mathlib's general existence and classification of fundamental units, where the old proof establishes the particular minimality directly. These theorems are already available through the original imports. The patch adds no axiom or import, but it does change which library results the proof uses. A project wishing to retain a visibly elementary minimality proof may prefer the original. The primary source for the reused results is mathlib source.

4 Reuse the existing odd-index theorem

There is a direct mathematical duplicate across `NativePellStepDown.lean` and `SignedPellResidue.lean`. The former proves that the auxiliary Pell index s is odd by assuming it even, calculating a residue $(-1)^h$, proving that the base divides one, and contradicting the lower bound on that base.

The latter already contains the precise reusable statement:

$$A > 1, \quad A \mid x_A(n) \quad \Longrightarrow \quad n \text{ is odd},$$

as `SignedPellResidue.odd_index_of_base_dvd`. After auxiliary classification the step-down proof has $Iu = x_I(s)$, so the needed divisibility is immediate. The replacement is:

```
have hsodd : Odd s :=
  SignedPellResidue.odd_index_of_base_dvd hI1
    (by rw [← hIu]; exact dvd_mul_right I u)
```

This reuses an existing result with exactly matching hypotheses. It needs no general theorem search tactic and no reformulation of the public step-down statement. The old local `hs0` proof of $s > 0$ is not

used later and is also removed. Its former input `hy` remains in the public signature, so existing callers retain the same interface.

The additional import is acyclic:

$$\texttt{NativePellStepDown} \longrightarrow \texttt{SignedPellResidue} \longrightarrow \texttt{Diophantine.Common.PellInt}.$$

In the combined patch the middle module also imports the new signed-congruence module described next. No reverse dependency is introduced.

The novelty distinction is narrow but useful. Earlier packets discuss casting whole Pell disjunctions and exposing source-register facts. This proposal instead removes a duplicate number-theoretic argument by using an existing odd-index theorem from a different file. It is an immediate adoption candidate even if the broader sign interface is deferred.

5 Compose congruence up to sign as a relation

5.1 The semantic operation is transitivity

The last part of `NativePellStepDown.index_residues` combines congruences that determine quantities only up to sign. The original proof introduces a local sign parameter, splits two alternatives against two more alternatives, extracts divisibility witnesses, and manufactures their sums or differences.

All this arithmetic expresses transitivity of the relation

$$a \sim_m b \quad \Longleftrightarrow \quad a \equiv b \pmod{m} \ \vee \ a \equiv -b \pmod{m}.$$

It is useful to give that relation a small explicit interface:

```
abbrev SignedModEq (m a b : ℤ) : Prop :=
  a ≡ b [ZMOD m] ∨ a ≡ -b [ZMOD m]
```

Symmetry follows by negating a congruence where necessary. Transitivity follows from the four sign products:

$$(+)(+) = +, \quad (+)(-) = -, \quad (-)(+) = -, \quad (-)(-) = +.$$

The helper proof performs those four cases once, using the existing `Int.ModEq.trans` and `Int.ModEq.neg` operations. Client proofs do not reopen integer quotient witnesses.

The new 43-line module provides symmetry, transitivity, conversion of $a \equiv \varepsilon b$ when $\varepsilon \in \{1, -1\}$, a divisibility characterization, and the absolute-value bridge $a \sim_m \mathrm{natAbs}(a)$. It is an abbreviation of the existing disjunction, so the public conclusions do not acquire a new wrapper type.

5.2 The exact step-down composition

The proof already has:

$$u \sim_c J, \qquad u \equiv (-1)^h s \pmod{c}, \qquad c \mid s - p \ \vee \ c \mid s + p.$$

Convert the middle fact to $u \sim_c s$, and the last fact to $s \sim_c p$. Then compose $J \sim_c u \sim_c s \sim_c p$. The delivered client is:

```
exact SignedModEq.iff_dvd_sub_or_add.mp
  (((SignedModEq.symm hu_J).trans
    (SignedModEq.of_sign_mul hu_cc (neg_one_pow_eq_or ℤ h))).trans
    (SignedModEq.iff_dvd_sub_or_add.mpr hsp'))
```

Two more clients consume the absolute-value bridge: `NativePellStepDown.signed_index_residues` and `SignedPellResidue.signed_residue_cases`. Their explicit positive/negative cases become composition with `SignedModEq.natAbs`.

The modulus is an arbitrary integer. No positivity hypothesis is hidden: for modulus zero, congruence becomes equality and the relation becomes equality up to sign; negative moduli also remain valid. The proof uses the unrestricted integer congruence API. Care is required in the divisibility bridge because `Int.modEq_iff_dvd` uses $m \mid b - a$. The helper explicitly reverses the congruences before simplifying $a - (-b) = a + b$. See mathlib source for these API conventions.

5.3 Charge the abstraction honestly

Taken together, odd-index reuse, removal of the unused local fact, and the signed-residue refactor change 350 lines in the two existing files into 292 lines there plus 43 lines of shared support: **335 lines total**, a net saving of 15. The signed relation by itself is primarily a structural simplification; its support cost absorbs much of the client saving.

The reason to adopt it is that sign ambiguity becomes an explicit operation with simple composition laws. The reason to defer it would be a preference for the smallest possible local patch when no further signed-residue clients are planned. The report does not count the parity saving a second time, and does not describe all 58 removed client lines as a net reduction.

6 Derive inverse pairing graphs by existential projection

6.1 One invertible graph has several coordinate views

The forward pairing relation already supplies all the mathematics needed by the inverse-coordinate representations. For natural pairing:

$$(\exists b,\ z = \mathrm{pair}(a,b)) \iff a = (\mathrm{unpair}(z))_1,$$
$$(\exists a,\ z = \mathrm{pair}(a,b)) \iff b = (\mathrm{unpair}(z))_2.$$

The new helpers prove these facts directly using the existing inverse laws. For example:

```
theorem exists_pair_right_iff (z a : ℕ) :
    (∃ b, z = Nat.pair a b) ↔ a = (Nat.unpair z).1 := by
  constructor
  · rintro ⟨b, rfl⟩
    simp
  · intro h
    exact ⟨(Nat.unpair z).2, by rw [h, Nat.pair_unpair]⟩
```

At the quadratic-certificate layer, `exists_certificate_iff` already relates the certificate to the forward pairing equation. Normalize the finite witness function using `Fin.exists_fin_succ_pi`; consume that existing equivalence; then apply the appropriate projection lemma. The proof does not need to build the quotient/slack vector again.

This applies to seven constructors across `PairingRepresentation.lean`, `QuadraticNatPairing.lean`, and `QuadraticTreeCodeGraphs.lean`: the two ordinary inverse coordinates, the quadratic forward graph, its two inverse coordinates, and the two tree-code coordinates. The tree case also uses the existing predecessor equation; the definition of the zero-code predecessor remains unchanged.

6.2 An actual quadratic client

The shortened left-coordinate constructor retains its system and witness dimensions exactly:

```
def leftRepresentation :
    Rep (Fin 2) (fun v => v 0 = (Nat.unpair (v 1)).1) :=
  (System.scopedRep (a := 2) (k := 4) (system 0 2)).ofIff (by
    intro v
    simp only [holds_iff]
    change (∃ w : Fin 4 → ℕ,
      Certificate (v 1) (v 0) (w 0) (w 1) (w 2) (w 3)) ↔ _
    simp [Fin.exists_fin_succ_pi, exists_certificate_iff,
      GraphRep.exists_pair_right_iff])
```

The seven declaration blocks shrink from 97 to 54 lines before support costs. The two projection helpers and their placement are charged to the complete module totals, which shrink from **396 to 369 lines**, a net saving of 27.

More importantly, all seven exact replacement constructors have passed equalities of the form

```
theorem leftRepresentation_short_eq :
    leftRepresentation_short = leftRepresentation := rfl
```

against the pinned original definitions. These equalities concern the full `Rep` values, including their expression and witness-count data. They are stronger than merely proving the same represented predicate.

This proposal is distinct from earlier witness-block transport and partial-decoder injectivity suggestions. The mathematical reuse is projection of the forward coding graph and elimination of its existing certificate. The finite-function theorem is a library tool used to expose that structure; see mathlib source.

7 Consume the Boolean certificate before constructing a conditional

`QuadraticBoolean.lean` already proves `exists_zero_certificate_iff`. Its zero-test and conditional representations nevertheless unpack certificate soundness and completeness and manually assemble witness functions.

For the zero test, normalize the one-coordinate witness function and apply that equivalence directly. For the conditional, normalize its two witnesses as a Boolean value b and a slack e. The selection equation

does not depend on e, so move the existential for e across that conjunct. The certificate equivalence then determines

$$b = \begin{cases} 1 & n = 0, \\ 0 & n \neq 0. \end{cases}$$

Substituting this value into the selection equation yields the desired conditional graph.

The final proof step is:

```
by_cases hn : v 1 = 0 <;>
  simp [Fin.exists_fin_succ_pi, exists_and_right,
    exists_zero_certificate_iff, hn, sub_eq_zero]
```

The existing two branches remain, but they are now the actual semantic branches of the conditional. There is no second manual proof of the zero-certificate specification and no hand-built witness tuple.

The complete module shrinks from **136 to 126 lines**, with no new production helper. Both actual replacement definitions compile, and both complete-value equality checks pass by `rfl`. Thus the zero system, conditional system, witness counts, and expression data are preserved.

This is not the earlier scanner Boolean-interface proposal: it concerns existential elimination in quadratic register representations, using a specification already present in the same file. It is also not a new Boolean encoding or a reduction in the number of Diophantine variables. The reduction is in proof construction.

8 Optional: recognize the physical permutation as a positive tail

This small proposal concerns `WangTiles942Permutation.lean`. Its existing zero and successor equations describe

$$\text{physicalPosition} = \text{Fin.cons}\big(0,\ i \mapsto 1 + \text{strictRank}(\text{nonStartWeight}, i)\big).$$

The zero head cannot equal a positive tail value, and the tail is injective by cancellation and the already-proved strict-rank injectivity. `Fin.cons_injective_iff` packages precisely those two facts.

The proposed proof is:

```
theorem physicalPosition_injective :
    Function.Injective physicalPosition := by
  have hform : physicalPosition =
      Fin.cons 0
        (fun i : Fin 941 => 1 + strictRank nonStartWeight i) :=
    funext (Fin.cases physicalPosition_zero physicalPosition_succ)
  rw [hform]
  exact Fin.cons_injective_iff.mpr ⟨by simp,
    fun i j h => strictRank_injective nonStartWeight_injective
      (Nat.add_left_cancel h)⟩
```

The preceding `nonStartWeight_injective` can also use composition directly:

```
theorem nonStartWeight_injective :
    Function.Injective nonStartWeight :=
  tile_weight_injective.comp (Fin.succ_injective 941)
```

The delivered file would shrink from 264 to 255 lines. The general zero-extension theorem with these exact mathematical ingredients has compiled successfully. The actual WangTiles942/K215 import chain was **not rebuilt**, so the full repository client remains a proposal pending its normal compilation gate. Its patch is in the optional directory and is not included in the main patch or 106-line total.

Earlier reports recognize the repository's existing recovered-index and strict-rank arguments. Those constructions are not claimed as new here. Only the specific final injectivity packaging and composition replacement are additional. The relevant library interface is in mathlib source.

9 Validation and its limits

9.1 Pinned environment and compilation

The verification used Lean 4.32.0, compiler commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. All nine dependency package commit hashes were checked against the pinned `lake-manifest.json`; mathlib is at `81a5d257c8e410db227a6665ed08f64fea08e997`. Dependency objects were reused from the corresponding cache, while **63 repository modules were compiled from the freshly downloaded pinned sources**. No previously built ProveIt objects from another snapshot were used as the repository baseline.

All **ten main proposed modules** were then compiled in a complete local import tree with the changed modules taking precedence. The finalizer pair, the three pairing modules, and the signed-residue/step-down/bootstrap chain were checked together in dependency order. The original working tree remained unchanged. This checks the complete changed source files, rather than only isolated snippets.

Check	Result and scope
Rational integral-power lemma	Passed; preserves the exact signed rational root.
Finalizer modules	Both complete revised modules passed together.
Small Pell criterion	Passed, including the original first-unit shape and sharpness witness.
Full Pell modules	Revised signed residue, step-down, and bootstrap all passed.
Pairing constructors	Seven exact clients and seven full-value `rfl` equalities passed.
Boolean constructors	Two exact clients and two full-value `rfl` equalities passed.
Complete representation files	All four revised modules passed.
Signed congruence interface	Complete new module passed; unrestricted integer modulus.
Permutation schema	Generic theorem passed; full WangTiles942 client not compiled.

The included logs retain compiler diagnostics and source hashes. Intermediate elaboration issues were corrected before the final successful compilations; the files and records in the final package correspond to those successful versions. Local elapsed times are recorded for reproducibility but are not used as before/after performance measurements.

9.2 What the representation equalities establish

The `Rep` constructors contain both mathematical correctness fields and computational data. Merely proving that a shortened constructor describes the same relation would not establish that it has the same witness count or literal expression.

The nine equalities compare each complete shortened constructor with its original and close by `rfl`. Lean's proof irrelevance permits the correctness proofs to differ, while the remaining data are definitionally equal. This gives a direct check on the concern that proof refactoring might silently change the Diophantine representation or size accounting.

Those checks run against the original snapshot before patch application. After the patch, the newly shared helper names are already present, so the original baseline-check files should not simply be rerun in the modified namespace. The replay instructions make this order explicit.

9.3 Axiom and interface review

An integrated `##print axioms` audit checks the shared lemmas, the altered proofs, and selected downstream endpoints for finalizer equivalence, Pell classification, and representation construction. The printed dependencies are drawn from the standard set

$$\{\texttt{propext}, \texttt{Classical.choice}, \texttt{Quot.sound}\}.$$

No `sorryAx` or new custom axiom is introduced. The old and new `firstUnit_fundamental` have the same printed axiom set. This is an audit of these checked declarations, not a statement that every possible repository theorem was audited.

Independent source review checks that all pre-existing declaration signatures are retained, that the added imports are acyclic, and that the finalizer norm and value definitions are unchanged. The new signed congruence is an abbreviation of an existing proposition. The patch does not change any published equation, register layout, witness dimension, or consistency hypothesis.

9.4 What has not been established

The whole repository and all 574 Lean files were not rebuilt. The user should run the repository's usual CI and endpoint checks when integrating the patch. The optional permutation client remains uncompiled against its complete dependency chain. No elaboration-time speedup, smaller kernel proof term, or improved Diophantine variable count follows from the source-line numbers.

The mathematical reformulations are checked in ordinary Lean arithmetic. This report makes no new claim about the equivalence between Diophantine non-solvability and Con(ZFC); it preserves the interfaces of existing proofs used in that program.

10 Adoption order and package guide

10.1 Recommended order

The lowest-coupling first step is the finalizer pair. It gives the largest net reduction, retains its imports, and replaces duplicated root-sign bookkeeping by a direct integral-root interface.

Next adopt the pairing and Boolean changes. They consume local mathematical specifications already present, and the nine full-value equality checks give strong evidence that computational data are unchanged.

The odd-index reuse can be adopted as a small direct cleanup. The main patch combines it with the signed-congruence module; review that abstraction on maintainability grounds, with its complete support cost visible. Adopt the small-ordinate Pell criterion if relying on the already-imported general Pell classification is consistent with the preferred exposition.

The permutation proposal is last: its generic argument is checked and its patch is small, but its full module must pass the normal project gate before adoption.

10.2 Archive contents

Path	Purpose
`report/`	This LaTeX source and its compiled PDF.
`patches/main.patch`	Combined checked changes, relative to the pinned repository.
`proposed/`	Complete replacement modules and the new signed-congruence module.
`checks/baseline/`	Original-snapshot constructor comparisons with nine `rfl` checks.
`checks/standalone/`	Reusable mathematical prototypes and the Pell sharpness example.
`checks/IntegratedAxiomAudit.lean`	Axiom queries for the combined modified modules.
`evidence/`	Source metrics, hashes, module-build records, and final compiler logs.
`audit/`	Prior-proposal exclusion map, archive/member hashes, and peer review.
`optional/`	Permutation patch and exact-client candidate, explicitly not fully compiled.
`scripts/`	Patch checking, replay, and archive-integrity verification helpers.

The root README contains the concrete commands. Before applying a patch, verify the commit and run `git apply --check`. The baseline constructor checks belong before application; the integrated axiom audit belongs after building the modified modules. The replay helper requires an explicit `--apply` argument before it changes the supplied checkout.

The proposed source files are included for inspection, but the patch is the integration mechanism. Do not independently copy files and then apply the same patch. The optional permutation patch does not overlap the main patch.

11 Further research questions

11.1 Can rational witness descent become a small common API?

The new lemma already handles every nonzero natural power, rather than only squares. Other proofs that create rational witnesses and later prove their powers integral may benefit from returning the original signed witness. A useful follow-up is to survey actual consumers before creating a broader abstraction. An integrally closed domain formulation is mathematically natural, but could introduce heavier imports than the elementary denominator proof. The decision should be based on repeated clients, not generality alone.

11.2 How far does the small-ordinate Pell interface extend?

The absolute threshold three is sharp. A more informative criterion could use additional lower bounds on a hypothetical fundamental unit, or a discriminant-dependent bound. The useful question is whether another existing minimality proof can consume such a criterion and offset its support cost. It would also be reasonable to keep both the elementary special proof and the short general proof if their different mathematical explanations serve different documentation needs.

11.3 Where else is sign ambiguity a compositional relation?

The new relation models the two units 1 and -1. More complicated root-of-unity ambiguity can be modeled by a finite group action, but this would be a different and more expensive abstraction. First identify existing proofs that already branch repeatedly on signs. Additional signed-residue clients are the most plausible way to turn the current structural improvement into a larger source reduction.

11.4 Can certificate consumers systematically preserve data by construction?

The pairing and Boolean examples suggest a focused review question: does a representation constructor reopen soundness and completeness of a certificate after an existential specification has already been proved? Such a client may be shortened without changing its system, expression, or witness count. Every candidate should retain a full-value equality check where possible. This guards against confusing a proof simplification with a different encoding that merely represents the same predicate.

11.5 Can future simplification reports compare maintained proof cost?

Large generated declarations can dominate raw line counts, while a small semantic API can remove more conceptual duplication. Future evaluations could record three separate quantities: complete maintained-source lines, the number of independent mathematical arguments, and measured elaboration/kernel-checking costs. This report establishes only the first quantity numerically and explains the second qualitatively. It supplies hashes and replay files so a controlled performance comparison can be run without conflating different repository snapshots.

A Novelty comparison at a glance

The full exclusion map is intentionally kept in the machine-readable and Markdown audit files. This table records the nearest existing ideas so that the distinction of each proposal can be reviewed directly.

Proposal	Nearby prior material	Additional content here
Exact rational-root descent	Cast cleanup and common scalar finalizer tails	Keep the actual rational root through denominator-one descent; remove four sign combinations and share the separated reconstruction theorem.
Small-ordinate Pell criterion	Pell cast bridges and named source-register facts	A sharp general $0 < y \leq 3$ fundamental-unit criterion with a checked counterexample at $y = 4$.
Odd-index reuse	Whole-disjunction cast simplifications	Use the existing `odd_index_of_base_dvd` theorem to delete a duplicate parity proof in another module.
Signed congruence composition	Ordinary modular/carry interfaces	Symmetry and transitivity of congruence up to sign, plus a natural absolute-value bridge.
Pairing graph projections	Witness-block transport, nine-coordinate packaging, partial-decoder injection	Eliminate a coordinate of the forward pairing graph through its inverse laws and existing quadratic certificate iff theorem.
Boolean certificate elimination	Scanner Boolean interfaces and tuple-layout suggestions	Consume `exists_zero_certificate_iff` before solving the quadratic conditional graph.
Physical-position injection	Existing recovered-index and strict-rank proofs	Use `Fin.cons_injective_iff` for the final zero-head, positive-tail map; full client remains optional and uncompiled.

The incoming README points to an `ADOPTION.md` file that is absent in the inspected commit. Its available placed report, README, placement note, and artifacts were inspected; no content is attributed to the missing file. Archive hashes in the supplied audit make the comparison reproducible.

B Primary source anchors

All project links in this document are pinned to the inspected commit. The principal targets are `NearSquareFinalizer.lean`, `MarginFinalizer.lean`, `NativePellBootstrap.lean`, `NativePellStepDown.lean`, `SignedPellResidue.lean`, `PairingRepresentation.lean`, `QuadraticNatPairing.lean`, `QuadraticTreeCodeGraphs.lean`, `QuadraticBoolean.lean`, and `WangTiles942Permutation.lean`.

The primary external APIs used in the new arguments are in the pinned mathlib modules mathlib source, mathlib source, mathlib source, mathlib source, and mathlib source. The Lean core denominator-power identity is checked using the exact toolchain recorded in the validation evidence. No changing web documentation is needed to replay the proofs.